Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность АС-68

Выполнил:

Студент 3-го курса

Группы АС-68

Ребенков А. Д.

Проверил:

А. А. Крощенко
Ст. преп. кафедры ИИТ,

«___»_____ 2026 г.

Брест 2026

Цель работы: выполнить моделирование прогнозирующей нелинейной ИНС.

Задание:

Вариант 8

№ варианта	a	b	c	d	Кол-во входов ИНС	Кол-во НЭ в скрытом слое
8	0.4	0.2	0.07	0.2	8	3

Для генерации обучающих и тестовых данных использовать функцию:

$$y = a * \cos(bx) + c * \sin(dx)$$

Код программы:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import pandas as pd

a, b, c, d = 0.4, 0.2, 0.07, 0.2
n_inputs = 8
n_hidden = 3

x = np.linspace(0, 50, 500)
y = a * np.cos(b * x) + c * np.sin(d * x)

X_data, y_data = [], []
for i in range(len(y) - n_inputs):
    X_data.append(y[i : i + n_inputs])
    y_data.append(y[i + n_inputs])

X_data = torch.tensor(np.array(X_data), dtype=torch.float32)
y_data = torch.tensor(np.array(y_data), dtype=torch.float32).unsqueeze(1)

split_idx = int(len(X_data) * 0.8)
X_train, y_train = X_data[:split_idx], y_data[:split_idx]
X_test, y_test = X_data[split_idx:], y_data[split_idx:]

class TimeSeriesNN(nn.Module):
    def __init__(self, in_features, hidden_features):
        super(TimeSeriesNN, self).__init__()
        self.hidden = nn.Linear(in_features, hidden_features)
        self.act = nn.Sigmoid()
        self.output = nn.Linear(hidden_features, 1)

    def forward(self, x):
        x = self.hidden(x)
        x = self.act(x)
        x = self.output(x)
        return x

model = TimeSeriesNN(n_inputs, n_hidden)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

epochs = 1500
loss_history = []

for epoch in range(epochs):
```

```
    model.train()
    optimizer.zero_grad()
    predictions = model(X_train)
    loss = criterion(predictions, y_train)
    loss.backward()
    optimizer.step()
    loss_history.append(loss.item())

model.eval()
with torch.no_grad():
    train_preds = model(X_train).numpy()
    test_preds = model(X_test).numpy()

plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
plt.plot(x[:split_idx+n_inputs], y[:split_idx+n_inputs], label="Участок
обучения", color='blue')
plt.plot(x[split_idx+n_inputs:], y[split_idx+n_inputs:], label="Участок
прогнозирования", color='orange')
plt.title("Прогнозируемая функция")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(loss_history, color='red')
plt.title("Динамика ошибки обучения")
plt.xlabel("Итерация")
plt.ylabel("MSE")
plt.grid(True)
plt.tight_layout()
plt.show()

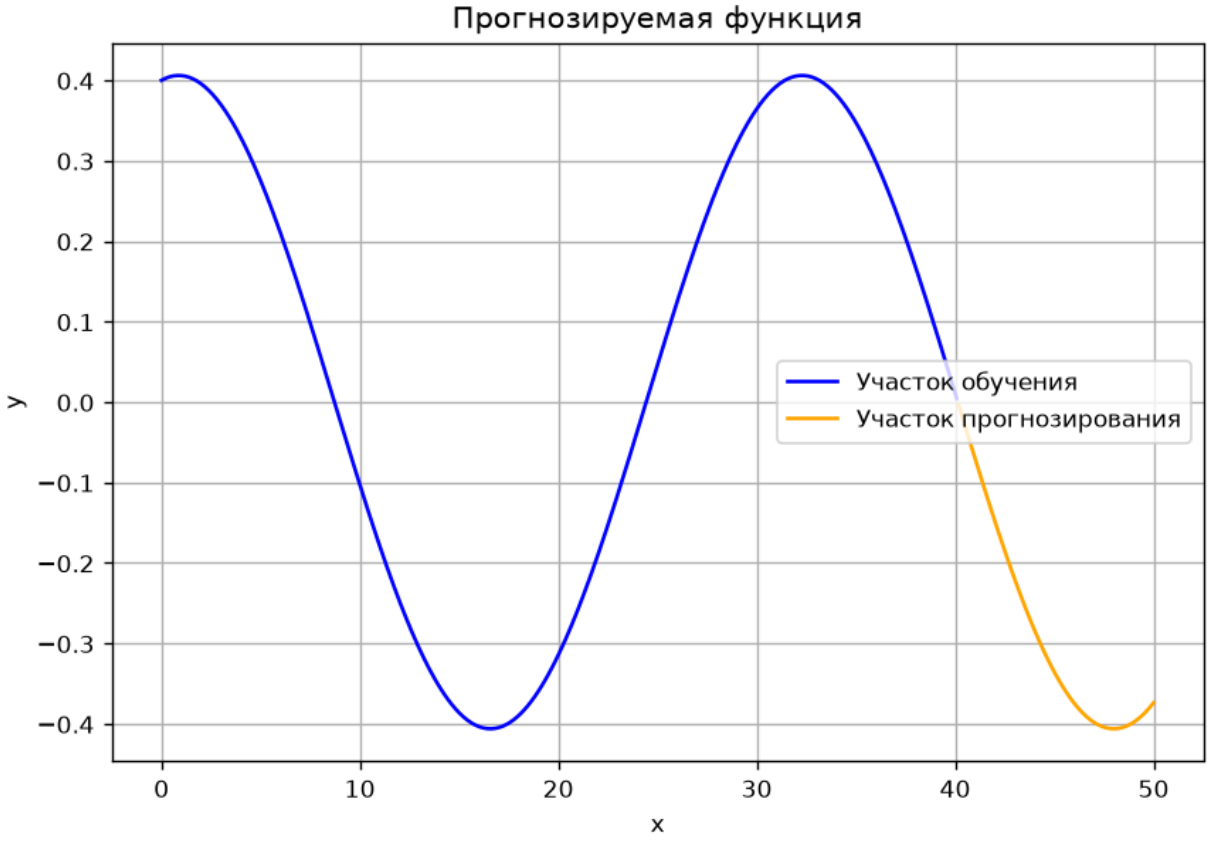
df_train = pd.DataFrame({
    'Эталонное значение': y_train.numpy().squeeze(),
    'Полученное значение': train_preds.squeeze(),
    'Отклонение': np.abs(y_train.numpy().squeeze() - train_preds.squeeze())
})

df_test = pd.DataFrame({
    'Эталонное значение': y_test.numpy().squeeze(),
    'Полученное значение': test_preds.squeeze(),
    'Отклонение': np.abs(y_test.numpy().squeeze() - test_preds.squeeze())
})

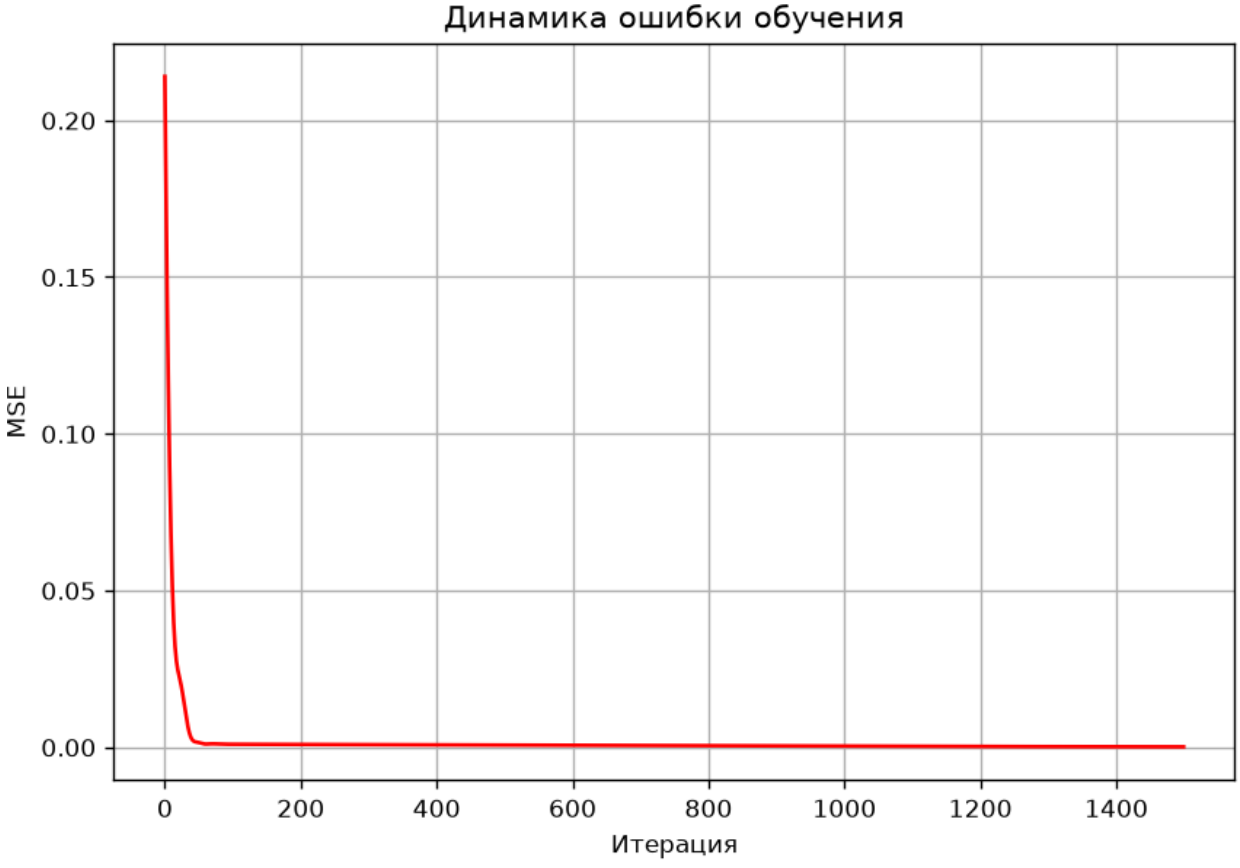
print("--- РЕЗУЛЬТАТЫ ОБУЧЕНИЯ ---")
print(df_train.to_string(index=False))

print("\n--- РЕЗУЛЬТАТЫ ПРОГНОЗИРОВАНИЯ ---")
print(df_test.to_string(index=False))
```

График прогнозируемой функции:



Изменение ошибки по эпохам:



Результат работы программы:

```
--- РЕЗУЛЬТАТЫ ОБУЧЕНИЯ ---
 Эталонное значение  Полученное значение  Отклонение
           0.406045              0.397722   0.008323
           0.406069              0.397869   0.008200
           0.405929              0.397876   0.008053
           0.405627              0.397744   0.007882
           0.405161              0.397474   0.007688
           0.404533              0.397064   0.007470
           0.403743              0.396514   0.007228
           0.402790              0.395826   0.006964

 --- РЕЗУЛЬТАТЫ ПРОГНОЗИРОВАНИЯ ---
  Эталонное значение  Полученное значение  Отклонение
           -0.003592             -0.000362   0.003230
           -0.011728             -0.008972   0.002756
           -0.019859             -0.017574   0.002286
           -0.027983             -0.026163   0.001820
           -0.036095             -0.034735   0.001361
           -0.044193             -0.043283   0.000910
           -0.052273             -0.051804   0.000470
           -0.060332             -0.060291   0.000041
           -0.068367             -0.068741   0.000373
           -0.076375             -0.077147   0.000772
           -0.084352             -0.085506   0.001154
           -0.092294             -0.093812   0.001518
```

Вывод: выполнил моделирование прогнозирующей нелинейной ИНС.